

Model Question Paper-1 with effect from 2022 (CBCS Scheme)

USN

--	--	--	--	--	--	--	--	--	--

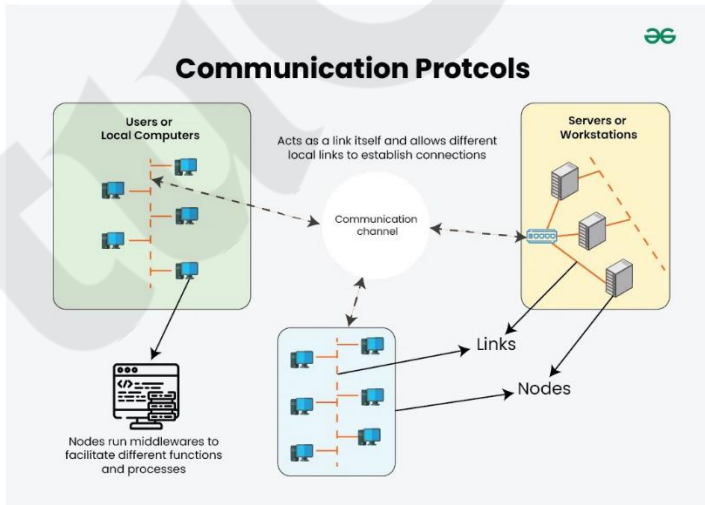
Sixth Semester B.E. Degree Examination

Cloud Computing

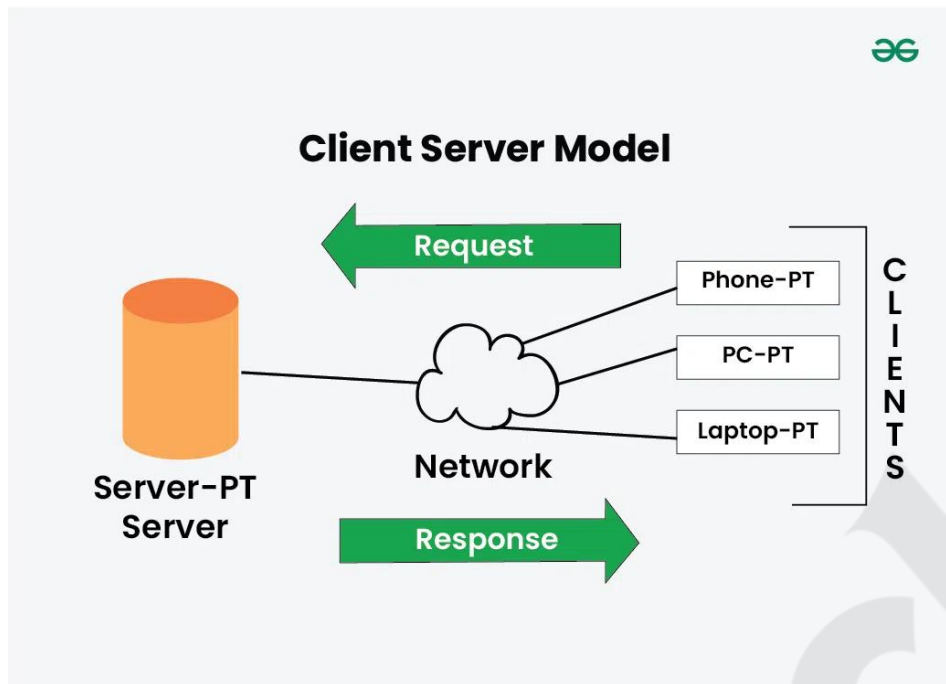
TIME: 03 Hours

Max. Marks: 100

Note: 01. Answer any **FIVE** full questions, choosing at least **ONE** question from each **MODULE**.

Module -1		Bloom's Taxonomy Level	Marks
Q.01	a Discuss in detail about distributed system models. Distributed System Models Distributed system models help in designing, analyzing, and understanding the behavior of distributed systems. They are categorized into Physical, Architectural, and Fundamental models.  1. Physical Model Represents the hardware layout of the system. • Nodes: Devices (servers, PCs) that process and communicate. • Links: Communication channels (wired/wireless) like point-to-point or broadcast. • Middleware: Software that enables communication, fault tolerance, synchronization.	L 2	10

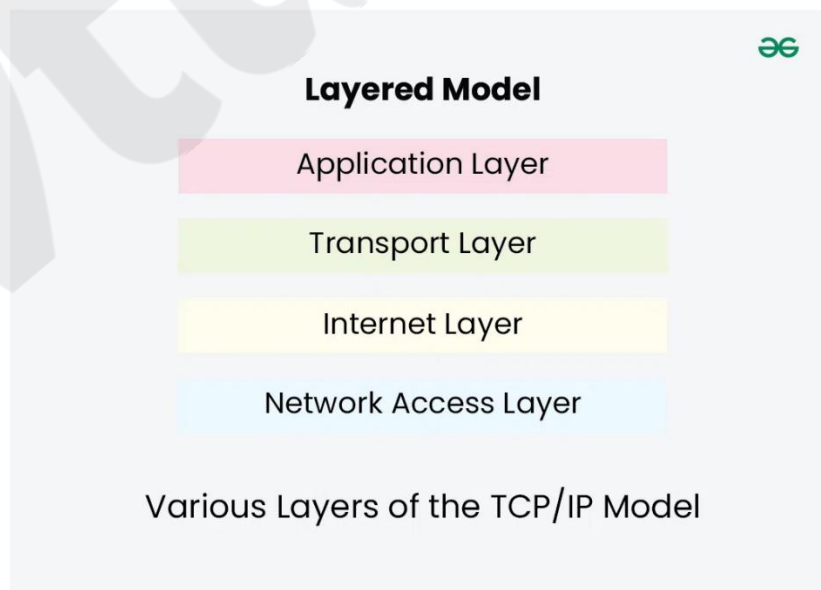
- **Network Topology:** Structure of node connections (bus, star, ring, mesh).
- **Protocols:** TCP, UDP, MQTT used for secure and efficient data exchange.



2. Architectural Model

Defines the system's organization and interaction patterns.

- **Client-Server Model:** Centralized server responds to client requests (e.g., web services).
- **Peer-to-Peer (P2P):** All nodes are equal and share services (e.g., BitTorrent).
- **Layered Model:** Organized into layers for modular design and abstraction.
- **Microservices Model:** Small, independent services performing specific functions, enhancing scalability.



3. Fundamental Model

Covers key concepts and formal behaviors.

- **Interaction Model:**

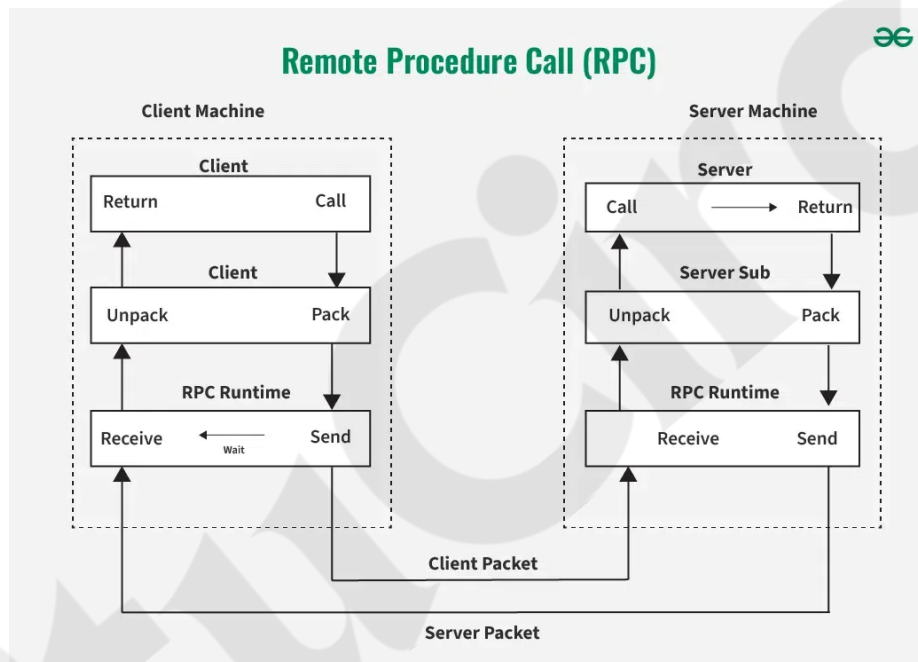
- *Message Passing*: Synchronous/asynchronous communication.
- *Publish/Subscribe*: Topics-based messaging.

- **Failure Model:**

- *Types*: Crash, omission, timing, Byzantine failures.
- *Handling*: Replication, fault detection, recovery methods.

- **Security Model:**

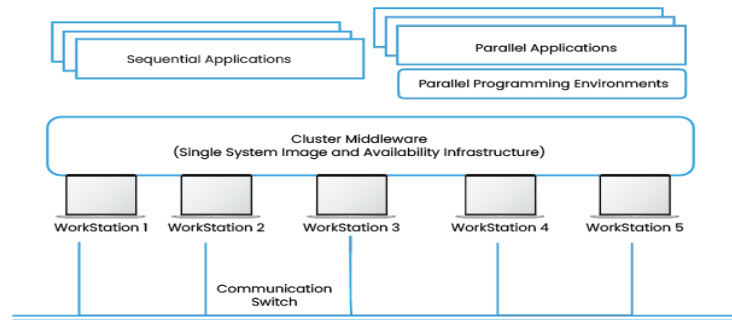
- *Authentication*: Passwords, keys, multi-factor verification.
- *Encryption*: Protects data confidentiality.
- *Data Integrity*: Hashing and digital signatures to prevent tampering.



b Explain the basic Cluster Architecture with a neat diagram.

L2

10



Cluster computing is a technique where **multiple interconnected computers (nodes)** work together as a **single system** to execute tasks, process data, or run applications. It provides users with a **transparent system** that appears as one virtual machine.

Features of Cluster Computing

1. **Transparency:** Users see a single virtual system instead of multiple nodes.
2. **Reliability:** Failure of one node doesn't affect the entire system.
3. **Scalability:** Nodes can be added or removed easily.
4. **Performance:** Parallel task execution improves overall speed.
5. **Load Balancing:** Tasks are distributed across nodes to prevent overload.

Cluster Computing Architecture

1. **Node (Computer)**
 - Each node has its own **processor, memory, and OS**.
 - Nodes are connected via a **high-speed network**.
2. **Head Node (Master Node)**
 - Manages the **cluster operations**.
 - Distributes tasks to other nodes (slave nodes).
 - Collects results and monitors performance.
3. **Slave Nodes (Worker Nodes)**
 - Execute the assigned tasks.
 - Report back to the head node.
 - Can perform computations in parallel.
4. **Cluster Middleware**
 - Software that handles **job scheduling, communication, and resource management**.
 - Examples: MPI (Message Passing Interface), OpenMPI, etc.

		5. Interconnect Network - Ensures high-speed data transfer between nodes. - Uses Ethernet, Infiniband, or other low-latency networks. 6. Storage System - Shared or distributed storage system (like NFS or parallel file systems). - All nodes may access the same data.		
OR				
Q.02	a	Write short notes on Peer-to-Peer network families. Definition - P2P architecture is a distributed model where each node (peer) acts as both client and server, sharing resources without a central authority. The diagram illustrates a Peer-to-Peer network architecture. It features four nodes, each represented by a laptop icon and labeled 'Node'. The nodes are arranged in a square pattern. Every node is connected to every other node in the network via bidirectional arrows, forming a fully connected mesh topology. This visualizes the decentralized nature of P2P networks where no single node acts as a central authority. 2. Characteristics Decentralization: No central server; peers communicate directly. Scalability: Easily grows to support more users. Fault Tolerance: Network survives even if some nodes fail. Resource Sharing: Peers contribute bandwidth, storage, and data. Autonomy: Each peer manages its own data and functions. 3. Types of P2P Networks Pure P2P: Fully decentralized (e.g., BitTorrent). Hybrid P2P: Uses central servers or super peers (e.g., Skype). Overlay P2P: Virtual network over physical internet (e.g., Chord). Structured P2P: Organized topology with routing rules (e.g., Pastry). Unstructured P2P: Random topology, no fixed structure (e.g., Gnutella).	L 2	10

		4. Components • Peer Nodes: Active devices in the network. • Overlay Network: Virtual layer connecting peers. • Indexing Mechanisms: Help locate shared resources. • Bootstrapping Mechanisms: Enable new peers to join the network. 5. Bootstrapping in P2P • Helps new peers discover others and connect. • Can use centralized servers, peer exchange, or DHTs. 6. Data Management • Storage: Distributed across peers. • Retrieval: Uses search algorithms. • Replication: Increases availability. • Consistency: Ensures all replicas are up to date. 7. Routing Algorithms • Flooding: Sends to all neighbors — high traffic. • Random Walk: Selects random paths — less overhead. • DHTs: Efficient lookups via hash tables (e.g., Kademlia). • Small-World Routing: Uses short paths and local/global links. 8. Advantages • No central point of failure • Efficient resource utilization • Cost-effective • High availability due to replication 9. Challenges • Difficult to scale with efficiency • Security risks from malicious nodes • Inconsistent content quality • Complex consistency and data management		
	b	Discuss system attacks and threats to cyberspace resulting in 4 types of losses.	L 2	10

1. Common System Attacks:

1. Malware Attacks:

- Includes viruses, worms, ransomware, spyware, and Trojans.
- Designed to steal, encrypt, or delete data or disrupt operations.

2. Phishing:

- Deceptive messages to trick users into giving up sensitive info like passwords or credit card numbers.

3. Denial of Service (DoS/DDoS):

- Overloads networks or servers, making them unavailable to users.

4. Man-in-the-Middle (MitM):

- Attackers intercept communication between two parties to steal or alter data.

5. SQL Injection:

- Injects malicious SQL queries into input fields to access or manipulate databases.
-

Four Types of Losses Due to Cyber Attacks

1. Financial Loss

- **(i)** Cyber attacks like ransomware or online fraud can lead to direct theft of money or demand for large payments.
- **(ii)** Organizations incur heavy costs for legal penalties, data recovery, and strengthening future security.

2. Data Loss

- **(i)** Attacks such as malware, hacking, or unauthorized access can result in loss or theft of sensitive data.
- **(ii)** Loss of intellectual property, customer information, or confidential business records affects compliance and trust.

3. Reputational Loss

- **(i)** A successful cyber attack damages an organization's public image and brand value.
- **(ii)** Customers may lose confidence, leading to a decline in user base and revenue.

4. Operational Loss

- **(i)** Cyber threats like Denial of Service (DoS) can bring down servers, disrupting business operations.

		• (ii) Delays in service delivery and system downtime reduce productivity and efficiency.		
Module-2				
Q. 03	a	Explain in detail about Implementation Levels of virtualization. 1. Instruction Set Architecture (ISA) Level Virtualization 1. Emulates a guest ISA on a host with a different ISA. 2. Allows execution of legacy or cross-platform binary code. 3. Achieved through code interpretation or dynamic binary translation. 4. Very flexible but has low performance due to instruction overhead. 5. Adds a software translation layer between compiler and processor. 2. Hardware Abstraction Level Virtualization 1. Virtualizes hardware directly using a hypervisor (e.g., Xen, VMware). 2. Provides virtual CPUs, memory, and I/O to guest OSs. 3. High performance due to close interaction with physical hardware. 4. Complex to implement and manage. 5. Enables running multiple OSs on the same physical machine. 3. Operating System Level Virtualization 1. Provides isolated user-space instances (containers). 2. Shares a single OS kernel across all containers. 3. Efficient resource use and fast startup. 4. Limited flexibility – all containers must use the same OS. 5. Suitable for lightweight server consolidation.	L2	10

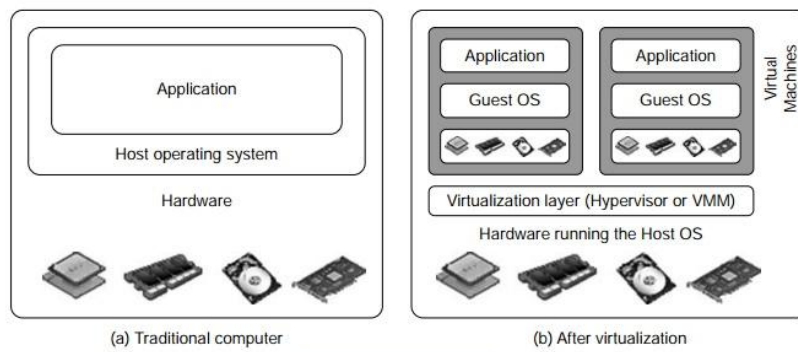


FIGURE 3.1

The architecture of a computer system before and after virtualization, where VMM stands for virtual machine monitor.

4. Library Support Level Virtualization

1. Virtualizes the API layer between apps and OS.
2. Allows apps to run in different environments (e.g., WINE for Windows apps on UNIX).
3. Less overhead than full system virtualization.
4. Not all applications may work correctly.
5. Useful for GPU virtualization (e.g., vCUDA).

5. User/Application-Level Virtualization

1. Virtualizes individual applications as isolated units.
2. Examples include JVM (.java) and .NET CLR (.NET apps).
3. Easy to deploy and portable across platforms.
4. Limited isolation compared to lower-level virtualization.
5. Used in sandboxing, application streaming, and secure app deployment.

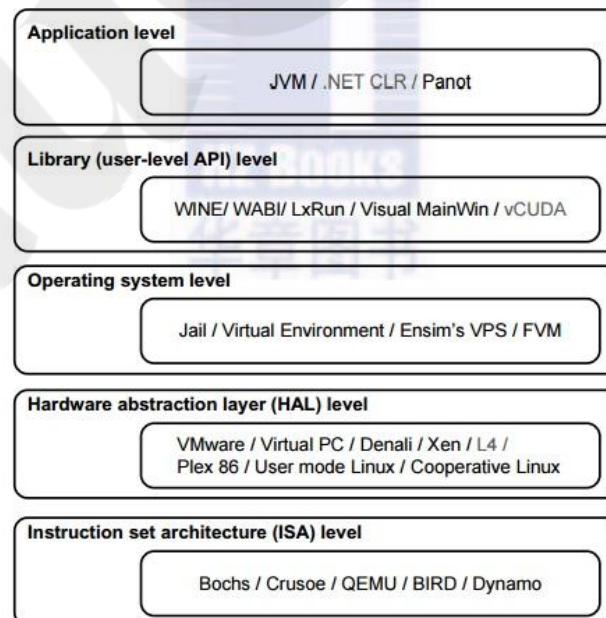


FIGURE 3.2

Virtualization ranging from hardware to applications in five abstraction levels.

b Explain how Migration of Memory, Files, and Network Resources happen in cloud computing.

2, 3

7

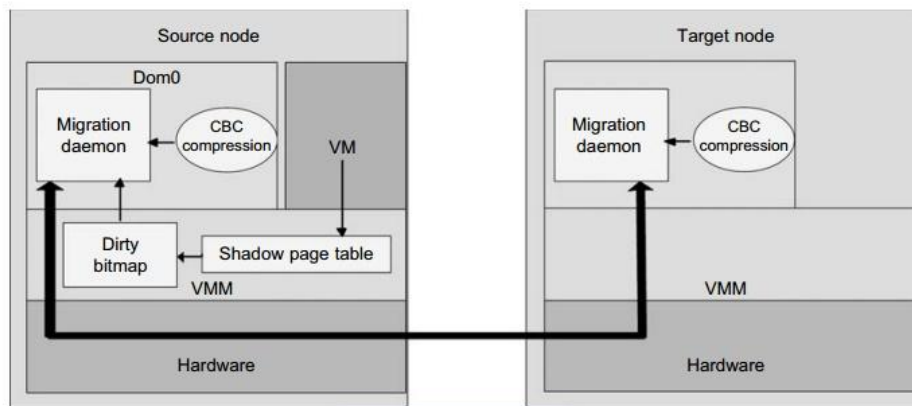


FIGURE 3.22

Live migration of VM from the Dom0 domain to a Xen-enabled target host.

1. Memory Migration

- Moves the VM's memory state from source to destination host.
- Internet Suspend-Resume (ISR) technique uses **temporal locality** to avoid redundant transfers.
- Tree-based file structures allow **only changed files** to be sent.
- ISR results in **high downtime**, suitable for non-live migrations.
- Efficient memory handling is essential due to **large size (MBs to GBs)** and need for speed.

2. File System Migration

- VMs need **consistent, location-independent** file systems on all hosts.
- Using a **virtual disk** per VM is simple but not scalable.
- **Global/distributed file systems** remove need for full file copying.
- ISR copies only the required VM files into the local file system.
- **Smart copying** and **proactive transfer** reduce data by using **spatial locality** and **prediction**.

3. Network Migration

- Migrated VMs must **retain all open network connections**.
- VMs use **virtual IP/MAC addresses**, independent of host hardware.
- **ARP replies** notify the network of new locations (on LAN).
- Live migration enables **no downtime**, with iterative **precopy** or **postcopy** techniques.
- Precopy allows continuous execution but may suffer **network load**; postcopy reduces data size but increases **downtime**.

		4. Live Migration Using Xen • Xen supports live VM migration with minimal service interruption. • Dom0 manages migration, using send/receive and shadow page tables. • RDMA enables fast transfer by bypassing TCP/IP stack and CPU. • Memory compression is used to reduce data size and overhead. • Migration daemons track and send modified pages based on dirty bitmaps.		
OR				
Q.04	a	Explain VM based intrusion detection system.  Importance of Intrusion Detection (ID) in Cloud • Detects and responds to attacks on systems and data. • Required by many security standards and regulations. • Must be integrated into any cloud deployment strategy.  Intrusion Detection by Cloud Service Model 1. Software as a Service (SaaS) • IDS responsibility: Provider • Customer role: Limited, may access logs for monitoring. 2. Platform as a Service (PaaS) • IDS responsibility: Provider • Customer role: Can configure app logs for external monitoring. 3. Infrastructure as a Service (IaaS) • IDS responsibility: Shared • Customer has flexibility to deploy IDS within VMs, networks, etc.  Where to Perform Intrusion Detection in IaaS 1. Within Virtual Machines (VMs) ○ Customer-managed HIDS ○ Detects activity inside VM. 2. At Hypervisor or Host Level ○ Provider-managed HIDS ○ Monitors VM-to-VM traffic and host behavior. 3. In Virtual Network ○ IDS monitors intra-VM and VM-host traffic (stays within hypervisor). 4. In Traditional Network	L2	10

	○ Provider-managed NIDS ○ Detects traffic entering or leaving the host system. 22 Responsibility Clarification • Providers: ○ Deploy and manage IDS (host, hypervisor, virtual network). ○ Must notify customers (via SLA) of any relevant attacks. • Customers: ○ Deploy HIDS inside VMs. ○ Integrate IDS into their monitoring systems. ○ Must negotiate visibility/alerts via contracts. 20 Types of Intrusion Detection Systems 1. Host-Based IDS (HIDS) • Runs on individual VMs (by customer) or host (by provider). • Monitors system activities and logs. • Challenge: Limited provider transparency for hypervisor HIDS. 2. Network-Based IDS (NIDS) • Monitors traditional network traffic. • Limitations: ○ Cannot inspect virtual network traffic. ○ Ineffective against encrypted traffic. 3. Hypervisor-Based IDS (via VM Introspection) • Monitors all inter-VM and VM-hypervisor communications. • Advantage: Full visibility. • Limitation: Complex, emerging technology; provider-managed.		
b	Write steps for Creating a Virtual Machine: Configure and deploy a virtual machine with specific CPU and memory requirements in Google Cloud. [or] Write 5 commands and explain Exploring AWS Cloud Shell. Step 1: Sign in to Google Cloud Console 1. Go to Google Cloud Console: https://console.cloud.google.com/ 2. Log in with your Google Account.	L2	7

3. Select or create a **new project** from the top navigation bar.

Step 2: Open Compute Engine

1. In the left sidebar, **navigate to "Compute Engine"** → Click **"VM instances"**.
2. Click **"Create Instance"**.

Step 3: Configure the Virtual Machine

1. Name the VM

- Enter a **name** for your VM instance.

2. Select the Region and Zone

- Choose a **region** close to your target audience or users.
- Choose an **availability zone** (e.g., us-central1-a).

3. Choose the Machine Configuration

- Under **"Machine Configuration"**, select:
 - **Series** (E2, N1, N2, etc.)
 - **Machine type** (Select based on your CPU & RAM needs)
 - Example:
 - **e2-medium** (2 vCPU, 4GB RAM)
 - **n1-standard-4** (4 vCPU, 16GB RAM)
 - Click **"Customize"** if you want specific CPU & RAM.

4. Boot Disk (Operating System)

- Click **"Change"** under Boot Disk.
- Choose an **Operating System** (e.g., Ubuntu, Windows, Debian).
- Select **disk size** (e.g., 20GB or more).

5. Networking and Firewall

- Enable **"Allow HTTP Traffic"** or **"Allow HTTPS Traffic"** if needed.
- Click **"Advanced options"** for networking configurations.

Step 4: Create and Deploy the VM

1. Review all the configurations.
2. Click **"Create"** to deploy the VM.
3. Wait for the instance to be provisioned.

Step 5: Connect to the VM

1. Using SSH (Web)

- Go to **Compute Engine** → **VM Instances**.
- Click **"SSH"** next to your VM instance.

2. Using SSH (Terminal)

- Open **Google Cloud SDK** (Cloud Shell) or your local terminal.
- Run:

```
gcloud compute ssh your-instance-name --zone=us-central1-a
```

Step 6: Verify and Use the VM

- Check CPU and Memory:

```
lscpu    # CPU details
```

```
free -h  # Memory details
```

- Install required software (example: Apache web server)

```
sudo apt update && sudo apt install apache2 -y
```

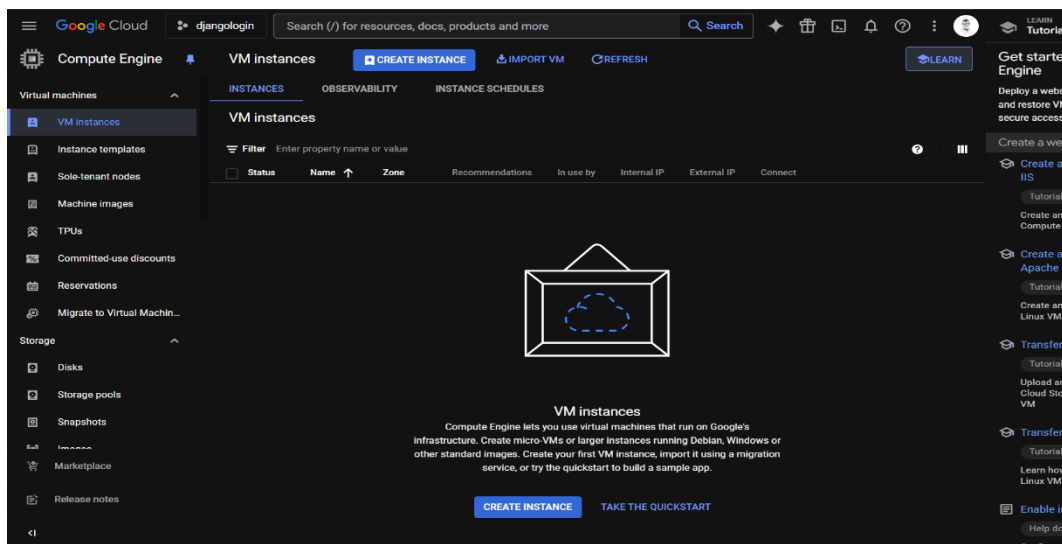
Step 7: Stop or Delete the VM (Optional)

- Stop the VM:

```
gcloud compute instances stop your-instance-name --zone=us-central1-a
```

- Delete the VM:

```
gcloud compute instances delete your-instance-name --zone=us-central1-a
```

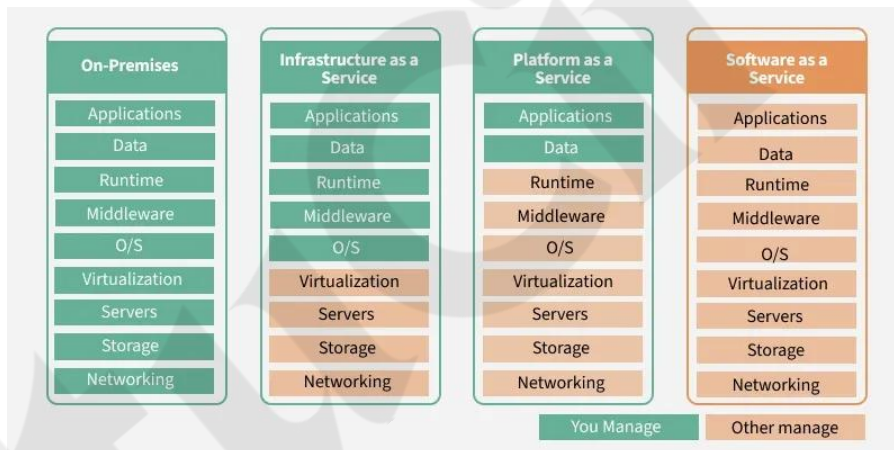


Module-3

Q. 05 a **Discuss IaaS, PaaS and SaaS cloud service models at different service levels.**

L2

10



☒ 1. Definition

- **IaaS (Infrastructure as a Service):** Provides virtualized computing resources like servers, storage, and networking.
- **PaaS (Platform as a Service):** Offers a development environment with tools to build, test, and deploy applications.
- **SaaS (Software as a Service):** Delivers fully functional software applications over the internet.

☒ 2. Users

- **IaaS:** Network architects, IT administrators, skilled developers.
- **PaaS:** Software developers and programmers.
- **SaaS:** End-users, business teams, consumers.

☒ **3. Technical Knowledge Required**

- **IaaS:** High technical knowledge.
- **PaaS:** Moderate coding knowledge.
- **SaaS:** No technical knowledge needed.

☒ **4. User Controls**

- **IaaS:** Full control (OS, runtime, middleware, applications).
- **PaaS:** Control over app and data only.
- **SaaS:** No control (everything managed by provider).

☒ **5. Examples**

- **IaaS:** AWS EC2, Microsoft Azure, Google Compute Engine.
- **PaaS:** Google App Engine, AWS Elastic Beanstalk, IBM Cloud.
- **SaaS:** Google Workspace, Salesforce, Zoom, Microsoft 365.

☒ **6. Use Cases**

- **IaaS:** Hosting websites, big data analytics, backup and recovery.
- **PaaS:** Developing web/mobile apps, APIs, microservices.
- **SaaS:** Email, CRM, video conferencing, document collaboration.

☒ **7. Cost and Scalability**

- **IaaS:** Pay-as-you-go model, highly scalable.
- **PaaS:** Cost-effective development platform, scalable.
- **SaaS:** Subscription-based, scalable for all business sizes.

☒ **8. Analogy (Food Example)**

- **SaaS:** You order and eat food (ready-to-use).
- **PaaS:** You bake a cake in a provided kitchen (need skills, but setup is done).
- **IaaS:** You rent a bare kitchen and cook from scratch (do everything yourself).

☒ **9. Cloud & Enterprise Services**

- **IaaS:** AWS VPC, vCloud Express.
- **PaaS:** Microsoft Azure, Google App Engine.
- **SaaS:** Google Apps, Facebook, MS Office Web, Salesforce.

☒ 10. Market Trend

- **IaaS:** ~12% growth.
- **PaaS:** ~32% growth.
- **SaaS:** ~27% growth.

b **Explain Private, public and hybrid cloud deployment models**

- Determines **where infrastructure is located** and **who owns/controls it**.
- Defines the **nature, purpose, and access** of the cloud environment.
- Helps organizations **choose the best approach** based on governance, cost, flexibility, security, scalability, and management.

☒ Types of Cloud Deployment Models

1. Public Cloud

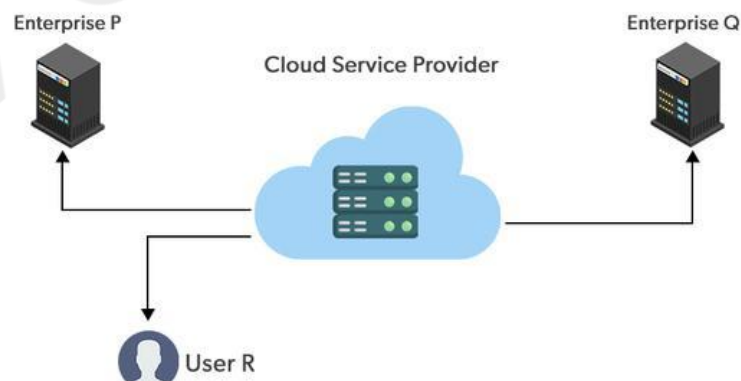
- **Open to all;** services are available to the general public over the internet.
- **Owned and managed** by third-party providers (e.g., Google Cloud, AWS).
- **Example:** Google App Engine

Advantages:

- Minimal investment (pay-as-you-go).
- No setup or infrastructure management.
- Maintenance handled by provider.
- Highly scalable on demand.

Disadvantages:

- Less secure (shared resources).
- Limited customization.



2. Private Cloud

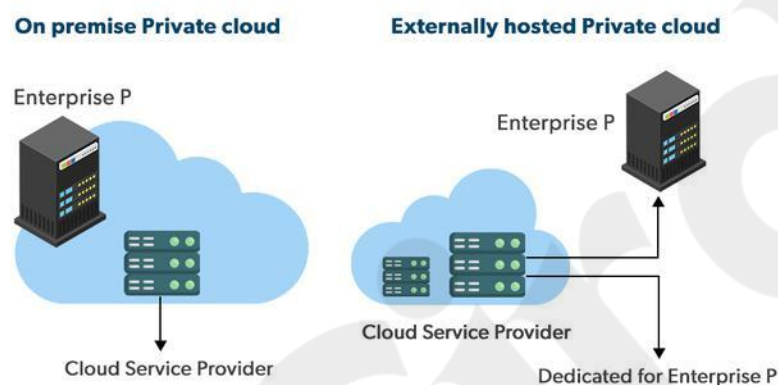
- **Used by a single organization**; exclusive access.
- Hosted on-premises or by a third party.
- Offers **greater control** and **security**.

Advantages:

- Full control over resources and policies.
- High data security and privacy.
- Supports legacy systems.
- Customizable for specific needs.

Disadvantages:

- Expensive to implement and maintain.
- Limited scalability compared to public cloud.



3. Hybrid Cloud

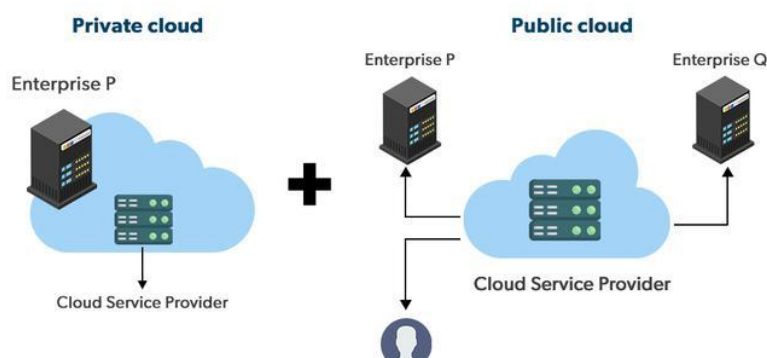
- **Combines public and private clouds** using proprietary software.
- Allows **data and apps to move between environments**.

Advantages:

- Flexible and customizable.
- Cost-effective (uses public cloud scalability).
- Better security with data segmentation.

Disadvantages:

- Complex to manage.
- Slower data transmission due to integration.



4. Community Cloud

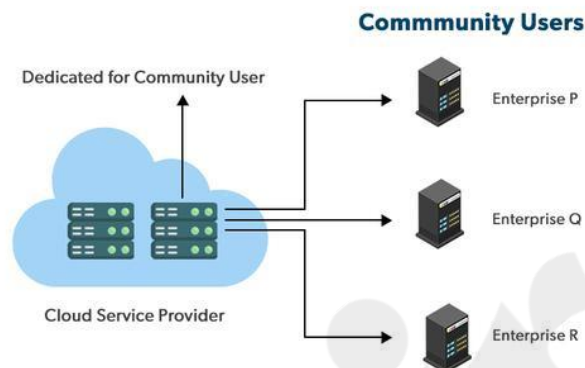
- Shared by **multiple organizations** with similar interests or concerns.
- Managed internally or by a third-party.

Advantages:

- Cost-effective due to shared resources.
- Good security and collaboration.
- Enables efficient data and infrastructure sharing.

Disadvantages:

- Limited scalability.
- Customization is difficult due to shared setup.



5. Multi-Cloud

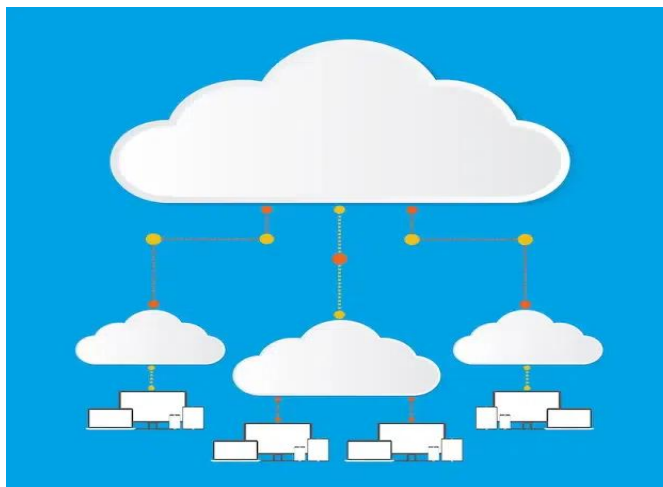
- Uses **multiple public cloud providers** simultaneously.
- Not limited to a single vendor or architecture.

Advantages:

- Mix and match best features of different providers.
- Low latency (choose nearest regions).
- High availability and fault tolerance.

Disadvantages:

- Complex architecture.
- Potential security risks due to integration gaps.



☑ Choosing the Right Cloud Deployment Model

Factors to Consider:

- **Cost** – Budget for infrastructure and service.
- **Scalability** – Ability to scale with growing demand.
- **Ease of Use** – Skill level required to manage the cloud.
- **Compliance** – Adherence to legal and regulatory standards.
- **Privacy** – Type and sensitivity of data being stored/processed.

➡ **No one-size-fits-all** – the best deployment model depends on current **business requirements**. You can switch models as your needs evolve.

OR

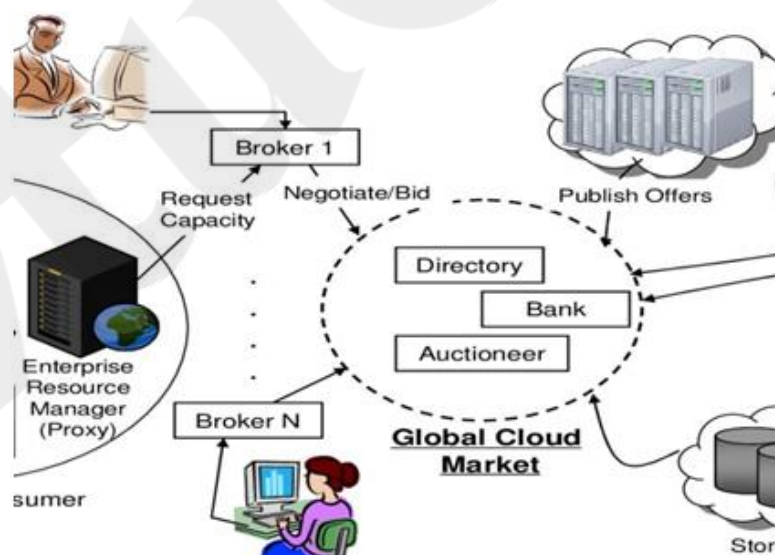
Q.
06

a **Write short notes on global exchange of cloud resources**

L2

10

- ❖ **Global Exchange of Cloud Resources** is the process of using cloud services in different parts of the world and countries.
- ❖ It allows businesses and organizations to deploy, manage, and grow their infrastructure all over the world.
- ❖ This process is made possible by cloud providers such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud, which provide data centers in different regions of the world.
- ❖ Such services enable organizations to provide resources cost-effectively, with little delay, and achieve high availability as well as regional compliance.



1. Geographical Distribution

- Cloud resources are hosted across a **network of global data centers** spread across various regions.
- This allows organizations to serve users from different locations with minimal delay, improving the overall user experience.

2. Load Balancing

- Cloud service providers offer **load balancing** across regions.
- This ensures that computing power and resources are efficiently distributed to meet fluctuating demands across different regions.

3. Redundancy and Availability

- The global exchange enables **redundancy** by hosting data in multiple locations.
- In the event of a system failure in one region, data and applications can still be accessed from other regions, ensuring **high availability**.

4. Latency Reduction

- By locating resources closer to the end-users, latency is reduced significantly.
- This enhances the performance of cloud-hosted applications, providing users with faster access to services regardless of their physical location.

5. Cost Efficiency

- **Pay-as-you-go** models and **cost-effective regional pricing** allow businesses to optimize their cloud expenditures.
- Companies only pay for the resources they use in specific regions, enabling better cost management.

6. Disaster Recovery

- The **global nature of cloud resources** ensures that businesses can implement effective **disaster recovery** strategies.
- By storing data across different regions, organizations can recover from outages in one region by switching to another region with no significant data loss or downtime.

7. Regulatory Compliance

- Many countries have strict **data residency** and **privacy laws**.
- The global distribution of cloud resources allows companies to adhere to **local regulations** by keeping data within the country or region where required.

Benefits of Global Exchange of Cloud Resources

1. Scalability:

- Global cloud resources can be scaled dynamically to handle varying demand.
- Companies can deploy additional resources across different regions based on performance needs.

2. **High Availability:**

- The global architecture of cloud resources ensures that services are available even during regional outages.
- Cloud providers offer multiple **availability zones** to support business continuity.

3. **Optimized Performance:**

- With resources located close to end-users, the **speed of access** is significantly improved.
- This is particularly important for applications requiring real-time data processing.

4. **Cost Management:**

- **Regional pricing** allows businesses to select the most cost-effective locations to host resources.
- This flexibility helps businesses minimize expenses and optimize their IT budgets.

Challenges of Global Exchange of Cloud Resources

1. **Data Privacy and Sovereignty:**

- **Data sovereignty** laws may limit the movement of data across borders.
- Compliance with local laws regarding data storage and privacy becomes complex when using a global cloud infrastructure.

2. **Network Latency:**

- Despite efforts to reduce latency, **network performance** between regions can still cause delays.
- This becomes a challenge for real-time applications that require minimal lag.

3. **Complexity in Management:**

- Managing **distributed cloud resources** across multiple regions can be complex.
- Businesses need advanced orchestration tools and skilled IT personnel to maintain performance and uptime.

4. **Security:**

- **Security risks** can increase with the complexity of managing resources across various regions.
- Data breach risks may be heightened due to multiple entry points and cross-border regulations.

	Examples of Global Cloud Providers 1. Amazon Web Services (AWS): ○ AWS has a vast network of data centers across the globe, supporting a variety of services such as EC2, S3, and RDS. ○ AWS ensures global scalability, high availability, and flexibility for businesses. 2. Microsoft Azure: ○ Azure operates data centers in over 60 regions, offering tools for deploying applications, managing data, and ensuring security. ○ Its global architecture supports businesses with complex regulatory and performance needs. 3. Google Cloud: ○ Google Cloud provides cloud services from numerous regions, allowing customers to deploy workloads worldwide. ○ Its global infrastructure offers low-latency access and high availability.		
b	Discuss a set of cloud services provided by Microsoft Azure. Cloud Services Provided by Microsoft Azure 1. Azure Compute Services ○ Azure Virtual Machines (VMs): Provides scalable computing resources on-demand for running Windows and Linux VMs. ○ Azure App Services: Managed platform for building and deploying web apps with support for various programming languages. ○ Azure Kubernetes Service (AKS): Simplifies containerized app management with Kubernetes orchestration. ○ Azure Functions: Serverless compute service for running event-driven functions without infrastructure management. ○ Azure Virtual Desktop: Desktop virtualization service for securely delivering remote desktop experiences. 2. Azure Storage Services ○ Azure Blob Storage: Scalable storage for unstructured data like text, images, and videos. ○ Azure Disk Storage: Persistent block-level storage for Azure VMs, offering different performance tiers. ○ Azure File Storage: Managed file shares accessible via the SMB protocol for shared storage.	L2	10

- **Azure Data Lake Storage:** Big data storage solution optimized for analytics with high scalability.
- **Azure Archive Storage:** Low-cost, long-term storage for infrequently accessed data.

3. Azure Networking Services

- **Azure Virtual Network (VNet):** Isolated cloud network for securely connecting Azure resources.
- **Azure Load Balancer:** Distributes incoming traffic across multiple servers to ensure high availability.
- **Azure VPN Gateway:** Securely connects on-premises networks to Azure using VPN.
- **Azure Application Gateway:** Layer 7 load balancer with web application firewall and URL-based routing.
- **Azure Content Delivery Network (CDN):** Delivers content faster to global users by caching data at edge locations.

4. Azure Databases and Analytics Services

- **Azure SQL Database:** Fully managed relational database service built on Microsoft SQL Server.
- **Azure Cosmos DB:** Globally distributed, multi-model database for high-performance applications.
- **Azure Synapse Analytics:** Analytics service combining big data and data warehousing capabilities.
- **Azure HDInsight:** Managed service for processing big data using open-source frameworks like Hadoop and Spark.
- **Azure Data Factory:** Cloud-based data integration service for moving and transforming data.

5. Azure Security and Identity Services

- **Azure Active Directory (Azure AD):** Identity and access management service for users and apps.
- **Azure Security Center:** Unified security management service for monitoring and protecting Azure resources.
- **Azure Key Vault:** Securely stores and manages keys, secrets, and certificates for apps.
- **Azure DDoS Protection:** Protection against distributed denial of service attacks, ensuring application availability.
- **Azure Information Protection:** Classification and encryption of data to prevent unauthorized access.

6. Azure DevOps and Developer Tools

- **Azure DevOps Services:** Cloud-based tools for managing the software development lifecycle, including CI/CD.
- **Azure DevTest Labs:** Helps create and manage test environments for development and testing.
- **Azure Container Instances (ACI):** Run containers without managing infrastructure.
- **Azure App Configuration:** Centralized management of configuration data for applications.
- **Azure Monitor:** Comprehensive monitoring solution for tracking performance, logs, and alerts.

7. Azure AI and Machine Learning Services

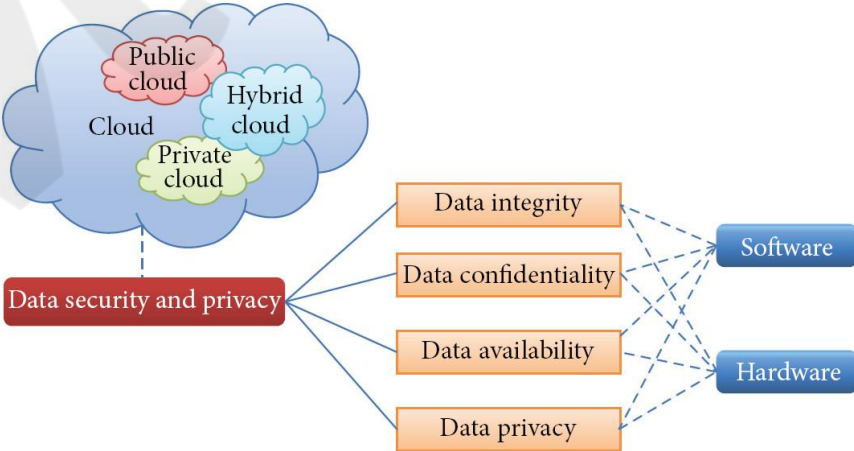
- **Azure Machine Learning:** Cloud service for building, training, and deploying machine learning models.
- **Azure Cognitive Services:** Pre-built APIs for vision, speech, language, and decision-making AI capabilities.
- **Azure Bot Services:** Platform for building intelligent bots and conversational interfaces.
- **Azure AI Gallery:** Repository for machine learning models, scripts, and solutions.
- **Azure Databricks:** Apache Spark-based analytics platform for data engineering and machine learning.

8. Azure Hybrid Cloud Solutions

- **Azure Arc:** Extends Azure management and services to on-premises, multi-cloud, and edge environments.
- **Azure Stack:** A set of hybrid cloud solutions that enable running Azure services on-premises.
- **Azure Site Recovery:** Disaster recovery service to ensure business continuity.
- **Azure ExpressRoute:** Private connection between on-premises data centers and Azure.
- **Azure Migrate:** Service to assess, migrate, and optimize workloads in the cloud.

9. Azure IoT Services

- **Azure IoT Hub:** Centralized platform for managing and connecting Internet of Things (IoT) devices.
- **Azure IoT Central:** Managed IoT app platform for simplifying IoT device management and analytics.

		○ Azure Digital Twins: A service for creating digital replicas of physical environments. ○ Azure Sphere: Securely connects microcontroller-powered devices to the cloud. ○ Azure Time Series Insights: Analytics platform for analyzing time-series data from IoT devices. 10. Azure Migration and Modernization Services ○ Azure Migrate: Tools and services for migrating on-premises workloads to Azure. ○ Azure Database Migration Service: Simplifies the migration of databases to Azure with minimal downtime. ○ Azure Web Apps Migration: Tool to migrate web apps from on-premises or other clouds to Azure. ○ Azure App Service Migration Assistant: Helps move .NET applications to Azure App Services. ○ Azure VMware Solution: Migrate VMware workloads to Azure without re-architecting applications.		
Module-4				
Q. 07	a	Discuss security of database services. Cloud Database Security refers to the strategies, technologies, and tools employed to protect cloud-hosted databases from unauthorized access, cyberattacks, data breaches, and other malicious threats. It ensures the integrity, confidentiality, and availability of data stored in cloud environments, and is essential for preventing data loss, exposure, and misuse. 	L2	10

Importance of Cloud Database Security

1. **Protection Against Cyber Threats:** As more enterprises migrate to the cloud, protecting sensitive data from hackers, malware, and unauthorized access becomes a significant concern.
2. **Governance and Compliance:** Maintaining regulatory compliance and meeting industry standards is crucial for avoiding legal repercussions and fines.
3. **Maintaining Customer Trust:** Proactive security measures ensure that customers' data is protected, helping businesses retain trust.
4. **Data Availability:** Cloud database security ensures that critical data remains accessible while preventing unauthorized disruptions.
5. **Business Continuity:** Effective security protocols are vital for ensuring the continuous operation of cloud services without unexpected downtime.

Features of Cloud Database Security

1. **Monitoring Database:** Utilizing customer-managed keys instead of relying on cloud providers for critical resource management to minimize third-party access risks.
2. **Manage Passwords:** Automating access control through password management systems to provide temporary passwords and permissions to authorized users.
3. **Logging Capabilities:** Enabling comprehensive logging to track unauthorized access attempts and storing logs for centralized security event management.
4. **Encrypted Database Access:** Enabling encryption for cloud databases to protect sensitive data and prevent unauthorized access.
5. **Access Control:** Defining strict access policies to limit who can access the database, ensuring that only authorized users have the appropriate permissions.

Advantages of Cloud Database Security

1. **Reduced Costs:** Cloud providers offer advanced security features that reduce administrative overhead, minimizing the total cost of ownership (TCO).
2. **Increased Visibility:** Enhanced security protocols allow businesses to monitor their data assets and user activity within the cloud environment.
3. **Native Applications Support:** Cloud databases provide native app integration without the need for additional installation, allowing developers to build seamless applications.

4. **Data Encryption:** Cloud services use sophisticated encryption to secure data both in transit and at rest, protecting sensitive information from unauthorized access.
5. **Automated Security Updates:** Cloud providers manage regular security updates and patches, ensuring the database is protected against emerging threats.

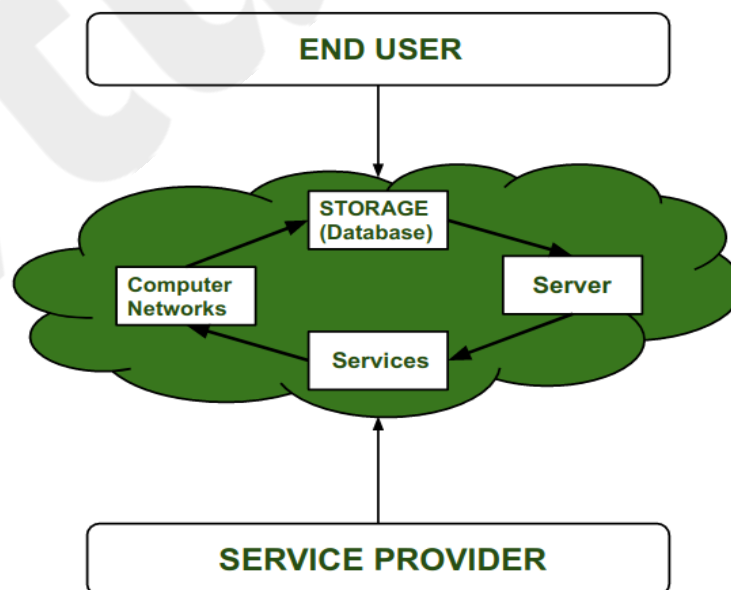
Disadvantages of Cloud Database Security

1. **Account Hijacking:** Attackers may use phishing or exploit vulnerabilities in third-party services to gain access to user accounts and expose sensitive data.
2. **Misconfiguration:** Cloud systems can become misconfigured over time as services expand, leaving gaps in security that attackers may exploit.
3. **Data Loss:** Unauthorized users may delete valuable data, causing irreparable loss, especially if backups are not managed securely.
4. **Data Breaches:** Inadequate security measures may lead to breaches, risking not only data but also the company's reputation and financial stability due to noncompliance fines.
5. **Shared Responsibility Model:** Cloud database security relies on both the provider and the user, with users needing to ensure proper configuration, monitoring, and access control.

b **Explain the security risks posed by shared images and management os.**

L2

10



Security Risks Posed by Shared Images:

1. Malicious Code Injection:

- Shared images can be pre-configured with malicious software that might go undetected during the creation or deployment of the image. When other users deploy the image, they might unknowingly execute this malicious code.

2. Unpatched Vulnerabilities:

- If the shared image is not updated regularly, it may contain outdated software with known vulnerabilities. This exposes the system to exploits and attacks.

3. Data Leakage:

- Sensitive data stored in a shared image may be accessible to other users or systems using the image. Improper data handling within shared images can lead to unauthorized data access.

4. Privilege Escalation:

- Shared images might contain embedded administrator or root privileges. If the image is not securely configured, it can allow unauthorized users to escalate their privileges and gain control of the system.

5. Lack of Isolation:

- In some cases, shared images may not have proper isolation between different users or virtual machines. This can lead to unintentional access to data or resources belonging to other users.

6. Compliance and Legal Risks:

- Shared images may not meet the required security and privacy standards for regulated industries. This poses a risk of non-compliance with laws such as GDPR, HIPAA, or PCI-DSS.

7. Insecure Configuration:

- Misconfigured settings in a shared image could lead to weak security controls, allowing attackers to exploit weaknesses in the system.

8. Inadequate Monitoring:

- Without adequate monitoring, it becomes difficult to detect suspicious activities related to shared images, such as unauthorized access or malicious activity.

Security Risks Posed by Management Operating Systems (OS):

1. Privilege Escalation and Unauthorized Access:

- If an attacker gains control of the management OS, they can escalate privileges and gain access to all virtual machines and systems managed by the OS. This can result in total control over the infrastructure.

2. Weak Authentication and Access Control:

- A poorly implemented authentication mechanism or lack of proper access control allows unauthorized users to access the management OS, putting all virtual environments at risk.

3. Denial of Service (DoS) Attacks:

- A compromised management OS can be used to perform DoS attacks on the virtual machines or containers, causing outages or performance degradation across all hosted services.

4. Insecure Communication:

- Communication between the management OS and other systems, such as virtual machines, could be intercepted if unencrypted protocols are used. This could expose sensitive data or allow attackers to tamper with communication.

5. Inadequate Resource Management:

- Poor resource allocation and management in the management OS can allow malicious users or processes to consume excessive system resources, leading to degraded performance or system crashes.

6. Exposure of Management Interfaces:

- The management OS often exposes interfaces for managing virtual machines or containers. If these interfaces are not secured, attackers may exploit them to compromise the system.

7. Unpatched Vulnerabilities:

- The management OS may contain vulnerabilities that can be exploited by attackers if not properly patched. This makes the OS a prime target for security breaches.

8. Insider Threats:

- Employees or individuals with access to the management OS may intentionally or unintentionally cause damage, leak data, or compromise system security.

9. Misconfigurations:

		○ Misconfigurations in the management OS can lead to vulnerabilities, including incorrect user permissions, weak passwords, or incorrect networking settings, all of which increase the risk of exploitation. 10. Lack of Auditing and Monitoring: • Without proper logging and monitoring, it becomes difficult to detect unusual activities or potential security breaches in the management OS, leaving the system vulnerable to attacks.		
OR				
Q. 08	a	Discuss how virtual machines are secured 1. Hypervisor Security • Ensure the integrity of the hypervisor through write protection and restricted access to prevent unauthorized modifications. • Implement isolation between VMs to prevent cross-VM attacks and intrusion detection to monitor hypervisor activity. 2. Virtual Machine Isolation • Enforce memory, network, and resource isolation to prevent unauthorized access between VMs. • Use strict access controls to limit communication and interactions between VMs. 3. Access Control and Authentication • Implement multi-factor authentication (MFA) and role-based access control (RBAC) to restrict access to VMs. • Maintain audit logs and enforce strong password policies to ensure only authorized access. 4. VM Monitoring and Logging • Continuously monitor VM behavior and maintain centralized logs for tracking potential security threats. • Set up real-time alerting to notify administrators of suspicious activities. 5. Guest Operating System and Application Security • Regularly update the guest OS and use security software like antivirus to protect against vulnerabilities. • Configure firewalls, IDS, and whitelisting to limit unauthorized access and application execution. 6. VM Image Security • Harden VM images before deployment and restrict image creation to trusted sources.	3, 4	10

		- Perform virus scanning on VM images to ensure they are free from malware or malicious content. 7. Data Encryption - Encrypt data at rest and in transit to protect sensitive information on VMs. - Use secure key management to ensure that encryption keys are properly managed and rotated. 8. VM Backup and Recovery - Perform regular backups and store them offsite to ensure data recovery in case of a breach. - Test disaster recovery plans to ensure VMs can be restored quickly after an incident. 9. Virtual Machine Patching and Updates - Apply automated patch management to ensure VMs are updated with the latest security patches. - Test patches in non-production environments before deployment to avoid disruptions. 10. VM Resource Management - Monitor VM resource usage to detect abnormal consumption patterns that could signal security threats. - Set resource allocation limits to prevent overuse by any single VM, maintaining performance and security.		
--	--	---	--	--

BCS502

	b	Explain reputation system design options. 1. Centralized Reputation System - A centralized system relies on a single authority or server to collect, store, and process reputation data for all users or services. Advantages: - Simplified management with a single point of control. - Easier to monitor and track user or service performance. Disadvantages: - A single point of failure can disrupt the entire system. - Potentially vulnerable to manipulation or attack if the central server is compromised.	L2	10
--	---	--	----	----

2. Decentralized Reputation System

- In this design, reputation data is stored and processed across multiple nodes, with no central authority. Each participant or service maintains their own reputation scores, and data is distributed among peers.
- **Advantages:**
 - Increased robustness since there's no single point of failure.
 - Better suited for distributed or peer-to-peer cloud environments.
- **Disadvantages:**
 - More complex to manage and ensure consistency across the system.
 - Higher computational and storage overhead as data needs to be distributed and verified across multiple nodes.

3. Hybrid Reputation System

- A hybrid system combines elements of both centralized and decentralized models. Typically, reputation data is stored centrally, but peer-to-peer evaluations or ratings are used to influence the final score.
- **Advantages:**
 - Flexibility in adapting to different cloud environments.
 - Provides a balance of reliability and robustness.
- **Disadvantages:**
 - May suffer from the complexity of managing multiple systems.
 - Still subject to the risks of centralization (e.g., targeted attacks).

4. Reputation Based on Feedback Mechanisms

- This system relies on user feedback or ratings after interacting with a service or user. Ratings from multiple users are aggregated to generate a reputation score for the service or user.
- **Advantages:**
 - Provides direct, real-time feedback from users, improving service accountability.
 - Scalable and adaptable to a wide range of cloud services.
- **Disadvantages:**
 - Susceptible to fake or biased feedback if not properly monitored or verified.
 - May require additional mechanisms (e.g., reputation decay) to ensure that scores remain relevant over time.

5. Reputation Based on Historical Behavior

- This system tracks the past behavior of users or services (e.g., uptime, reliability, or security events) and uses this data to predict future behavior.

The reputation score is dynamically updated based on ongoing performance metrics.

- **Advantages:**
 - Provides a continuous, data-driven evaluation of trustworthiness.
 - Reduces the impact of individual malicious actions since it focuses on long-term patterns.
- **Disadvantages:**
 - Requires large volumes of data and historical tracking, leading to increased storage and processing overhead.
 - May not quickly adapt to sudden, drastic changes in behavior.

6. Trust Models in Reputation Systems

- Trust models use algorithms or mathematical models to assign trustworthiness scores. These models often factor in various metrics, including past interactions, feedback, and service performance.
- **Advantages:**
 - Can be customized based on the needs of the specific cloud environment (e.g., service reliability, data integrity).
 - Provides a formal, quantifiable approach to reputation management.
- **Disadvantages:**
 - Complex to design and implement.
 - May need continuous refinement and updates to remain effective as the cloud environment evolves.

7. Reputation Based on Third-party Evaluation

- In this approach, a trusted third-party organization (e.g., an auditor or certification body) evaluates the reputation of services or users in the cloud.
- **Advantages:**
 - Enhances credibility as the third-party evaluation is independent.
 - Useful for situations requiring external verification, such as compliance with industry standards.
- **Disadvantages:**
 - Potentially slow and expensive due to the need for external evaluation.
 - May introduce a bottleneck if the third-party organization becomes overwhelmed with requests.

Module-5				
Q. 09	a	What are the various system issues for running a typical parallel program in either parallel or distributed manner? 1. Communication Overhead • Parallel systems (e.g., using threads or processes) may have lower communication latency due to shared memory. • Distributed systems must send data over a network, leading to higher latency and bandwidth constraints. 2. Synchronization and Coordination • Ensuring that multiple processes or threads coordinate properly is critical. • Problems like race conditions, deadlocks, and livelocks can occur. • Need for locks, barriers, semaphores, or message passing mechanisms. 3. Data Distribution and Locality • How data is divided among processes affects performance. • In distributed systems, poor data locality can result in excessive remote data access, hurting efficiency. 4. Load Balancing • Uneven workload distribution causes some nodes/threads to be idle while others are overloaded. • Requires dynamic or static load balancing strategies. 5. Fault Tolerance and Reliability • In distributed systems, nodes or network links can fail. • Systems must handle failures gracefully (e.g., checkpointing, replication). 6. Scalability • The ability of the system to maintain performance as more resources are added. • Communication, synchronization, and data contention may limit scalability. 7. Resource Management • Effective use of CPU, memory, network, and storage. • In distributed systems, resource heterogeneity (e.g., different hardware capabilities) complicates management.	L2	10

8. Programming Model Complexity

- Writing efficient parallel/distributed programs is harder.
- APIs like **MPI, OpenMP, CUDA, or MapReduce** help but require expertise.

9. Security Issues (Distributed Systems)

- Data transmission over networks introduces concerns about **data integrity, confidentiality, and authentication.**

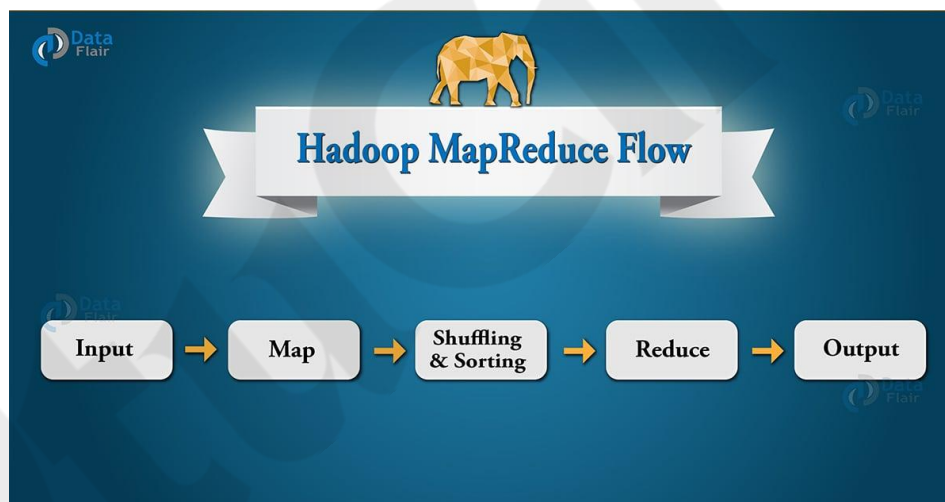
10. Debugging and Profiling

- Much harder than in sequential systems.
- Tools are needed for **monitoring, profiling, and debugging** parallel and distributed executions.

b **With a neat diagram explaining the data flow in running a MapReduce job at various task trackers using Hadoop Library**

L2

10



- ❖ Data locality is a principle in Hadoop that promotes moving computation (algorithms/code) close to where the data resides, instead of moving large data to computation.
- ❖ Designed to reduce the network congestion and enhance the performance of big data processing.

Step by step MapReduce Job Flow

The data processed by MapReduce should be stored in **HDFS**, which divides the data into blocks and store distributedly,

Below are the steps for MapReduce data flow:

- **Step 1:** One block is processed by one mapper at a time. In the mapper, a developer can specify his own business logic as per the requirements. In this manner, Map runs on all the nodes of the cluster and process the data blocks in parallel.
- **Step 2:** Output of Mapper also known as intermediate output is written to the local disk. An output of mapper is not stored on HDFS as this is temporary data and writing on HDFS will create unnecessary many copies.
- **Step 3:** Output of mapper is shuffled to reducer node (which is a normal slave node but reduce phase will run here hence called as reducer node). The shuffling/copying is a physical movement of data which is done over the network.
- **Step 4:** Once all the mappers are finished and their output is shuffled on reducer nodes then this intermediate output is merged & sorted. Which is then provided as input to reduce phase.
- **Step 5:** Reduce is the second phase of processing where the user can specify his own custom business logic as per the requirements. An input to a reducer is provided from all the mappers. An output of reducer is the final output, which is written on HDFS.

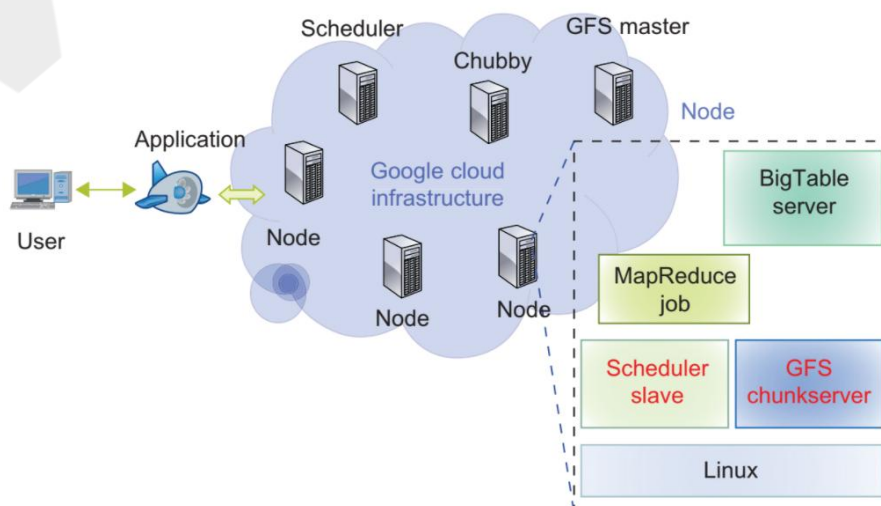
Hence, in this manner, a map-reduce job is executed over the cluster. All the complexities of distributed processing are handled by the framework. For example, data/code distribution, high availability, fault-tolerance, data locality, etc. The user just needs to concentrate on his own business requirements and write his custom code at specified phases (map and reduce).

OR

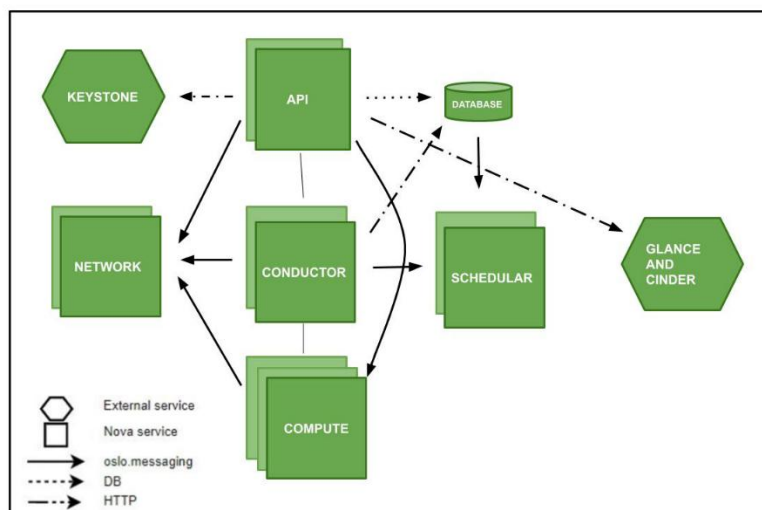
Q. 10 a **Discuss Programming the Google App Engine.**

3, 4

10



- | | | |
|--|--|--|
| | <ul style="list-style-type: none">• Google App Engine (GAE) is a fully managed Platform as a Service (PaaS) used for building and hosting scalable web applications on Google's infrastructure.• It dynamically scales web applications as traffic demand changes, ensuring efficient resource usage and high availability.• GAE supports multiple programming languages like Python, Java, Go, and PHP, each with its own runtime and SDK for local development and testing.• The App Engine SDK allows developers to emulate the production environment on local machines and later deploy their apps easily with cost-control quotas.• GAE provides numerous in-built services including cron jobs, queues, scalable datastores (Cloud SQL, Datastore, Memcached), communication tools, and in-memory caching.• It offers a secure and high-performance execution environment with general features (e.g., datastore, logs, blobstore, search) covered by service-level agreements (SLA).• GAE has preview and experimental features (e.g., Sockets, MapReduce, Prospective Search, OpenID) that may change and are accessible to selected users.• Third-party services and helper libraries are integrated via partnerships, enabling apps to perform extended tasks beyond core functionalities.• Key advantages include fast deployment, ease of use, rich APIs, built-in security, automatic scaling, high reliability, platform independence, and reduced infrastructure cost.• Overall, Google App Engine simplifies the development of robust, scalable, and secure applications without managing server infrastructure, making it ideal for rapid development and enterprise-scale solutions. | |
|--|--|--|



1. **Nova** is an OpenStack component responsible for managing and provisioning **virtual machine (VM) instances**, similar to **AWS EC2**, but for private clouds.
2. It supports multiple **hypervisors and virtualization technologies** such as **KVM, Hyper-V, VMware, and Xen**.
3. Nova interacts with other OpenStack services like:
 - **Keystone** for authentication,
 - **Glance** for image services,
 - **Neutron** for network provisioning,
 - **Cinder** for providing volumes to VM instances.
4. Nova is developed using **Python**, and uses libraries like:
 - Eventlet for networking,
 - SQLAlchemy for database interactions.
5. It follows a **horizontally scalable architecture**, meaning the workload is distributed across multiple servers instead of relying on a single machine.
6. Nova uses **SQL databases** to store information, which are shared logically by its components.
7. It operates using **multiple daemons** running on top of Linux servers, each performing specific tasks.
8. **Use Cases** of Nova include:
 - Creating and managing VMs,
 - Supporting bare-metal servers,
 - Offering limited support for system containers (e.g., Docker).
9. **Core components** in Nova architecture include:
 - **DB:** Central SQL database,
 - **API:** Handles HTTP requests and interacts with other components,
 - **Scheduler:** Allocates instances to hosts,

		○ Compute: Manages VMs and hypervisors, ○ Conductor: Coordinates complex tasks and acts as a DB proxy. 10. Users can access Nova services via: ○ Horizon (OpenStack dashboard UI), ○ CLI (Command Line Interface), ○ Novaclient (Python-based API and CLI tool for Nova operations).		
--	--	--	--	--

StudyCircle